Document Number: <QMS Ref. No.>	***Copy Number:*** <copy number>

TATA CONSULTANCY SERVICES

Trivandrum

vILP – STRUTS

Content Manual

Version 1.0

May 2014

DOCUMENT RELEASE NOTICE

Notice No.
Client
Project
Document details

Name	Version No.	Author	Description
vILP_Struts 2.0.doc	1	Anu Venugopal (726128)	Initial draft version of Struts 2.0 content

Revision details:

Action taken (add/del/chg)	Preceding page No.	New page No.	Revision description

Change Register serial numbers covered:
The documents or revised pages are subject to document control.
Please keep them up-to-date using the release notices from the distributor of the document.
These are confidential documents. Unauthorised access or copying is prohibited.

Approved by : _____ Date: _____

Authorised by: _____ Date: _____

DOCUMENT REVISION LIST

Client
Project
Document Name
Release Notice Reference (for release)

Rev. No.	Revision date	Revision description	Page No.	Prev page No.	Action taken	Addenda /New page	Release Notice reference

Course 2 - STRUTS 2.X.....4

2.1 Objective **4**

1.1 Course Content **4**

1.1.1 Application Environment Set-up using Struts 2.x 4

1.1.1.1 Main elements of Struts 2.x application 4

1.1.1.2 Struts 2.x application development step-by-step 5

1.1.1.3 “Hello World” application using Struts 2.x 5

1.1.1.4 Introduction to the main tags in struts.xml 13

1.1.1.5 Struts 2.x Life Cycle Process 14

1.1.1.6 Value Stack and OGNL (Object Graph Navigation Language).....16

1.1.1.7 ValueStack.....16

1.1.1.8 Other Stacks.....17

1.1.1.9 Object Graph Navigation Language (OGNL).....17

1.1.1.10 Struts 2.x Tag Library.....18

1.1.1.10.1 Introduction to JSP Custom Tags 18

1.1.1.10.2 Advantages of using JSP Custom Tags 18

1.1.1.10.3 How to incorporate Custom Tags inside the JSPs 18

1.1.1.11 Struts 2.x Tags in detail 19

1.1.2 UI Tags 19

1.1.2.1 Sample Scenario 1.....21

1.1.2.2 Sample Scenario 2.....22

1.1.2.3 Sample Scenario 3.....26

1.1.2.4 Sample Scenario 4.....35

1.1.2.5 Control Tags 40

1.1.2.6 If Else Tag with Scenario 40

1.1.2.7 Iterator Tag with Scenario.....43

1.1.2.8 Merge Tag with Scenario 45

1.1.2.9 Sort Tag with Scenario 48

1.1.2.10 Generator Tag with Scenario 53

1.1.2.11 Data Tags.....55

1.1.2.12 Property Tag.....56

1.1.2.13 Set Tag.....56

1.1.2.14 Bean Tag.....56

1.1.2.15 Push Tag.....57

1.1.2.16 Include Tag.....57

1.1.2.17 Param Tag 57

1.1.2.18 Putting all together (Scenario).....57

1.1.2.19 Recap 61

COURSE 1 - STRUTS 2.X

1.1 Objective

- Application Environment Set-up using Struts 2.x
- Struts 2.x Life cycle process
- Struts 2.x tag library
- Struts 2.x tags in detail

1.2 Course Content

1.2.1 Application Environment Set-up using Struts 2.x

1.2.1.1 Main elements of Struts 2.x application

A Struts 2.x application consists of,

i) Set of JSP pages
   - contains UI components to input/output the data.

ii) Set of action classes
   -holds the state of the UI components defined the corresponding jsp pages.

iii) Application configuration resource file
   - struts.xml

iv) A deployment descriptor
   - web.xml

v) Tags representing various components
   - UI components
   - Validators and converters

1.2.1.2 Struts 2.x application development step-by-step

Step 1: Create a dynamic web project

Step 2: Add the required jar files to the application's class path

Step 3: Configure the web.xml as given below,

Define an entry org.apache.struts2.dispatcher.StrutsPrepareAndExecuteFilter should be defined inside web.xml.This filter is the entry point to the struts2 web application.

Step 4: Create the Action classes

Step 5: Configure struts.xml

Step 6: Design the Views

Step 7: Test your application

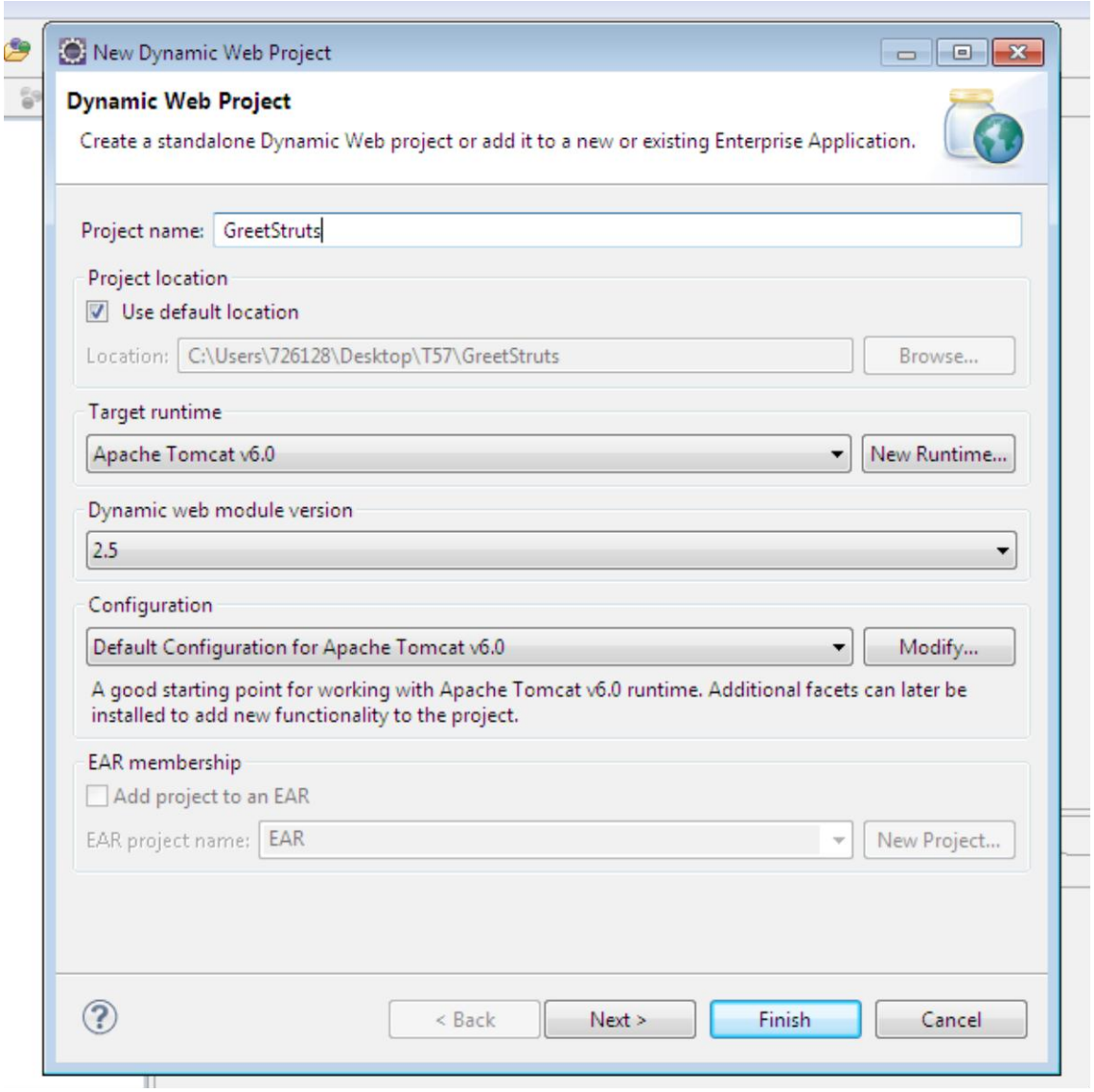
1.2.1.3 "Hello World" application using Struts 2.x

Objective: To print "Hello World" in the browser using Struts 2.x

Getting Started:

Step1: Create a Dynamic Web Project

Open Eclipse , go to File -> New -> Project and select **Dynamic Web Project** in the New Project wizard and write a name for the project for example **GreetStruts** and then press finish button

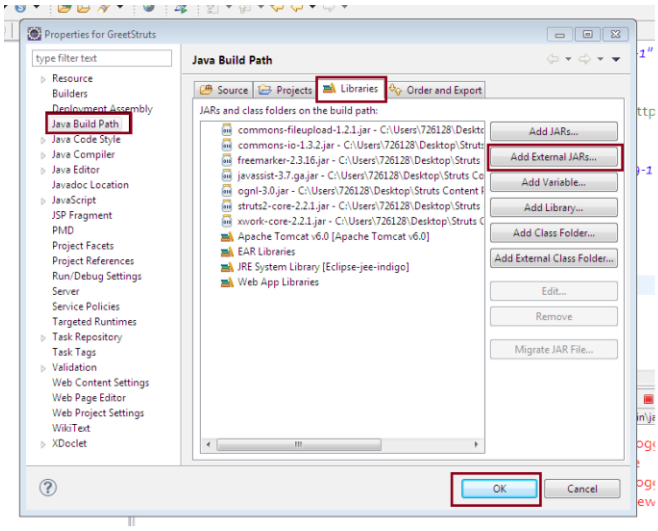


Step 2: Add the required jar files into the class path

Right click your project ie.GreetStuts->Build path->Configure Build path-->press the libraries tab->press add external jars button->select your destination folder where your jar files resides and then press ok

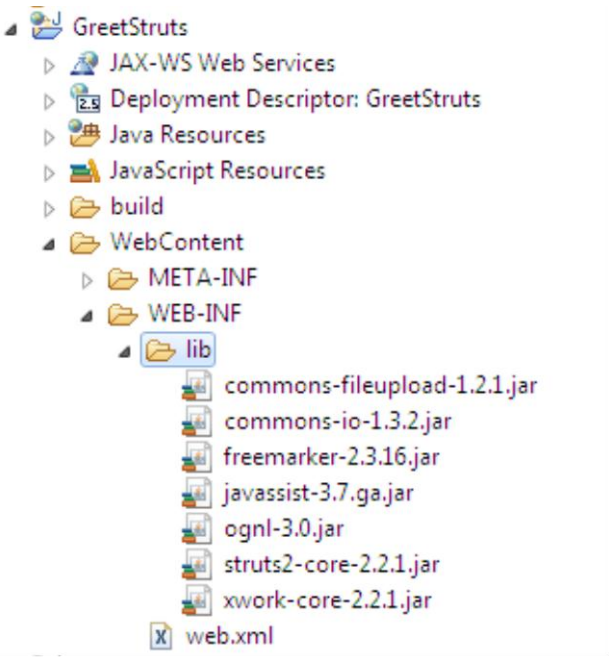
The required jar files are

1. commons-fileupload-1.2.1
2. commons-io-1.3.2
3. freemarker-2.3.16
4. javassist-3.7.ga
5. ognl-3.0
6. struts2-core-2.2.1
7. xwork-core-2.2.1



Note: There is another way to configure the jar files (preferred)

You can copy all the required JAR files in Web Content -> WEB-INF -> lib folder



Step 3: Mapping struts 2.x in web.xml

Struts has to be to be plugged into your web application to handle the incoming requests for processing. This is done by adding a filter definition in web.xml file. This filter essentially redirects all incoming requests to **StrutsPrepareAndExecuteFilter** which then use configurations to handle that request.

Open web.xml file which is under WEB-INF folder and copy paste following code

```xml
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns="http://java.sun.com/xml/ns/javaee"
  <display-name>GreetStruts</display-name>
  <welcome-file-list>
    <welcome-file>index.html</welcome-file>
    <welcome-file>index.htm</welcome-file>
    <welcome-file>index.jsp</welcome-file>
    <welcome-file>default.html</welcome-file>
    <welcome-file>default.htm</welcome-file>
    <welcome-file>default.jsp</welcome-file>
  </welcome-file-list>
  <filter>
    <filter-name>struts2</filter-name>
    <filter-class>org.apache.struts2.dispatcher.ng.filter.StrutsPrepareAndExecuteFilter</filter-class>
  </filter>
<filter-mapping>
    <filter-name>struts2</filter-name>
    <url-pattern>/*</url-pattern>
  </filter-mapping>
  </web-app>
```

Step 4: Configuring Action classes(s): put all business logic here

This is simple bean class. In struts 2, action is POJO (Plain Old Java Object). It has one extra method execute i e. Invoked by struts framework by default.

Note:

In Struts1 Action class is required.It is done by extending the org.apache.struts.action.Action class. But Struts 2 Action class is optional, but you are still allowed to implement the com.opensymphony.xwork2.Action for some handy constant values or extend the **com.opensymphony.xwork2.ActionSupport** for some common default Action implementation functions.

We need to create a action class (it is a simple POJO class, which will have one additional method called **execute**.) which will have the logic to print **"Hello World"**

Create a class under the package com.test.greet.GreetingStruts

GreetingStruts.java

```
package com.test.greet;
public class GreetingStruts {

  private String message;

  public String getMessage() {
    return message;
  }

  public void setMessage(String message) {
    this.message = message;
  }
public String execute() throws Exception {

  this.message="Hello World Welcome to Struts 2";
  return "success";
  }

}
```

1) The access specifier of the execute method should be public.
2) The return type of the execute method should be String

Step 5: Configure Struts.xml

This file contains all of the routing and configuration information for the Struts application.

To create struts.xml file,

Right click the src folder ->new->other->general->file->give file name as struts.xml.

Copy the following content into struts.xml.

```
struts.xml
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE struts PUBLIC
  "-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
  "http://struts.apache.org/dtds/struts-2.0.dtd">
<struts>

  <constant name="struts.enable.DynamicMethodInvocation" value="false" />
  <constant name="struts.devMode" value="true" />

  <include file="example.xml"/>

  <package name="test" namespace="/" extends="struts-default">

   <default-action-ref name="greeting"/>

   <action name="greeting" class="com.test.greet.GreetingStruts">
    <result name="success">/jsp/Greet.jsp</result>
   </action>
  </package>
</struts>
```

Root element of struts.xml

This tag represents a module of the application.

Action tag is the sub element of package tag. Inside the result tag only, the navigation rule will be defined.

Sub element of action tag. Inside the result tag only, the navigation rule will be defined.

In the above struts.xml, we have defined **GreetingStruts action of our application.** The navigation rule is set inside struts.xml using result tag. The navigation paths are mapped with GreetingStruts depending on the outcome of **execute ()** method. If **execute () method returns success**, user will be redirected to **Greet.jsp**

Note: Our GreetingStruts contains the method **execute()** which is the default method getting called by Sturts2. If the name of method is different, e.g. checkName(); then we should specify the method name in**<action>** tag

```
<action name="greeting" class="com.test.greet.GreetingStruts" method="checkName">
```

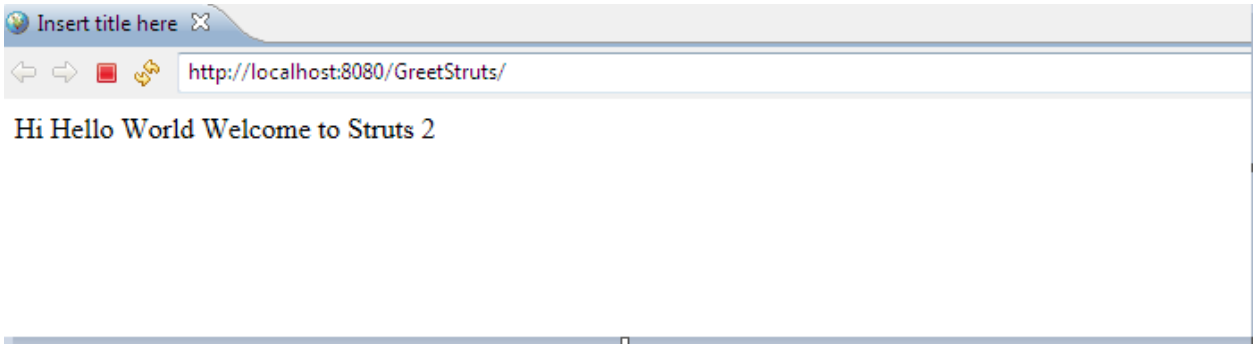
Step 6: Design your view (Greet.jsp)

```
Greet.jsp
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
  pageEncoding="ISO-8859-1"%>
<%@ taglib prefix="s" uri="/struts-tags" %>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>Insert title here</title>
</head>
<body>
Hi <s:property value="message"/>
</body>
</html>
```

Struts 2 tags will be included in the JSP page using taglib directive. URI indicates the location of taglibrary.prefix indicates the prefix of the struts 2 tag.

The value of message attribute GreetStruts action class will be rendered in the JSP page using property tag.More about property tag will be explained later.

Step 7: Test your application



1.2.1.4 Introduction to the main tags in struts.xml

i) The core configuration file for the framework is struts.xml.

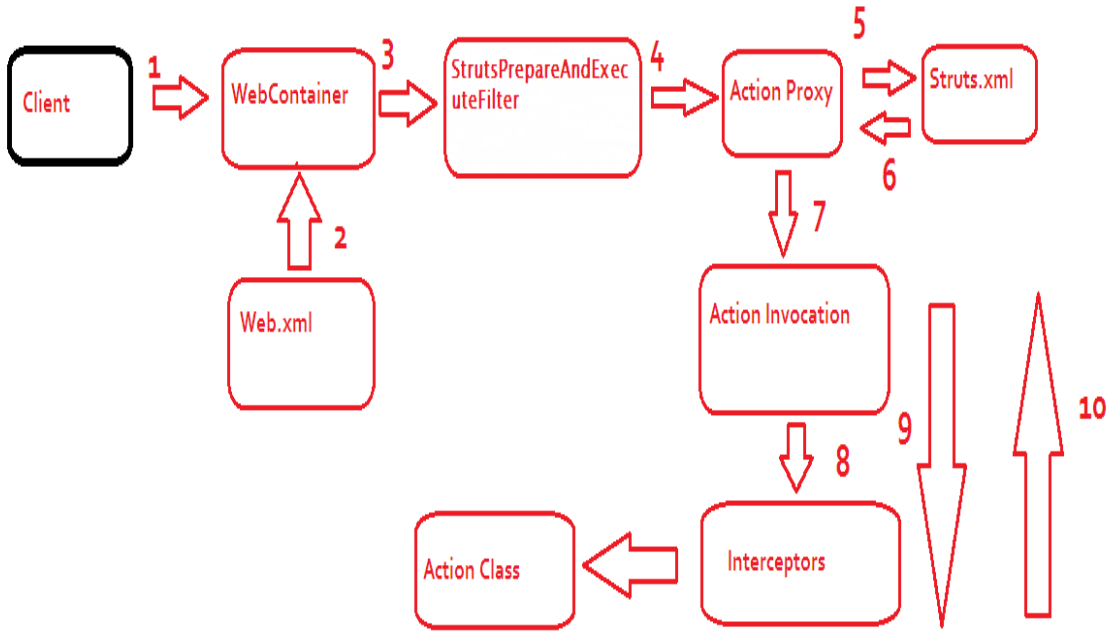
ii) It is the important file from where struts framework gets information about the action and decides which view to be invoked

Some important tags of struts.xml file are

Tag Name	Description
<constant name="struts.devMode" value="true" />	**Struts 2 has a setting** called devMode.When this setting is enabled, Struts 2 will provide additional logging and debug information, which can significantly speed up development. By default devMode value is false.
<struts>	Root Element of struts.xml
<package>	This tag represents a module of the application. Package tag has got an important attribute called,namespace. Namespace attribute is using for separating the web application modules.Using namespace, we can access these modules using the module names in the URL ex)**http://localhost:8080/strutsapp/workflowmanagement/createRegionalWorkflow**
<action>	Sub element of package tag. It represents an action to be called for an incoming request.
<result>	Sub element of action tag. Inside the result tag only, the navigation rule will be defined.

Note: struts.xml will be explained in detail later sections

1.2.1.5 Struts 2.x Life Cycle Process



1) The web browser sends a request to the container for a particular response.

2) Container loads the web.xml and checks for a matching URL Pattern.. If there is a match , container transfer the request to StrutsPrepareAndExecuteFilter

3) StrutsPrepareAndExecuteFilter identifies the suitable action by calling the ActionMapper internally, which will be calling ActionProxy.

4) ActionProxy reads the configuration file struts.xml and finds the suitable action for the request

5) ActionProxy creates an instance of ActionInvocation class. The ActionInvocation class invokes the Interceptors one by one (if required , and will be explaining later)

6) Interceptor methods will be invoked first before the control is taken over by action class. Inside the action class the requested method will be executed. ActionInvocation receives the final result produced by an action class.

7) ActionInvocation is responsible for looking up the proper result associated with the Action result code mapped in struts.xml.

8) StrutsPrepareAndExecuteFilter selects an appropriate view, basing on the result. The result is then sent back to the servlet container which in turn sends it back to the client. Then the result can be seen on the output of the browser in HTML, PDF, images or any other format.

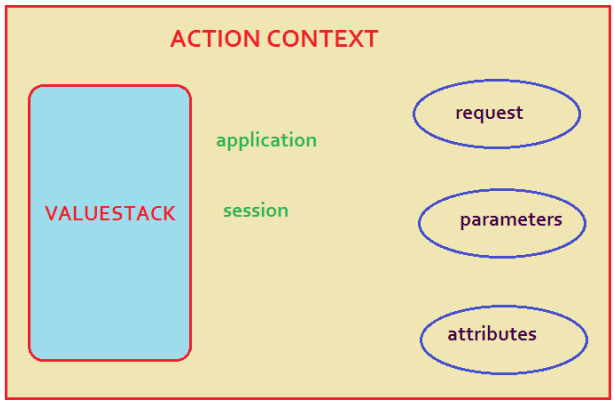
1.2.1.6 Value Stack and OGNL (Object Graph Navigation Language)

Two important feature of Struts 2 are ValueStack & OGNL.

1.2.1.7 ValueStack

A ValueStack is simply a stack that contains application specific objects such as action objects and other model object (Java Beans). At the execution time, action is placed on the top of the stack.

In Struts 2 the action resides on the ValueStack which is a part of the ActionContext. ActionContext is a global storage area that holds all the data associated with processing of a request.



1.2.1.8 Other Stacks

Application, session, request and param are the other stacks in which the in which application data are stored depending on the scope.

Application: Context level (Global) data's are stored in this stack

Session: Session level data's are stored in this stack

Request: request data's are stored in this stack

Parameter: All parameters are stored in this stack.

1.2.1.9 Object Graph Navigation Language (OGNL)

OGNL stands for Object Graph Navigation Language. This is an expression language used in Struts 2.OGNL to store parameters as java bean variables in VALUESTACK and to store the values from ValueStack in the result pages.

OGNL can be used for

i) OGNL can be used to bind GUI elements to model objects in struts framework.

ii) It can be used to invoke methods of the model.

iii) It can create lists, maps to be used with GUI, bind generic tags with model objects, and convert one type to another.

OGNL has got other two important functionalities.

i) Data Transfer

ii) Type Conversion

Struts 2 OGNL receives the request parameters from the request and transfer it to the respective java beans. By default the request parameters are String, but java bean variables can be String, int, Array or list or any other objects. There OGNL takes care of type conversions through its built in converters. Again when the results are generated. OGNL does the reverse ie it converts the property type to strings which then is used to generate HTML output

For example, if you have a property called user ID in an action class, LoginAction, then it can be referenced in jsp using struts tags

<s:property value="userId"/>

1.2.1.10 Struts 2.x Tag Library

1.2.1.10.1 Introduction to JSP Custom Tags

A tag is some text enclosed between the characters "<" and ">". Tags have names, attributes and values.HTML Tags are pre-defined.

Custom tags are user defined tags. For example, if you write <TABLE>, the browser understands that a table has to be displayed. However, if you want to print copy right information about your company in all your web pages, you cannot write something like <COPYRIGHT>. The browser will not understand the tag, because it is not a pre-defined HTML tag.

A Custom tag is a tag you write in your Java Server Page .When the JSP compiler compiles that tag, it will be rendered as proper HTML.

1.2.1.10.2 Advantages of using JSP Custom Tags

i) Custom tags reduce the use of scriplets inside the JSP pages.

ii) Using custom tags, we can separate the business logic from the JSP page, so that it will be easy to maintain.

iii) Once a custome tag is created, it can be used in all JSP files

1.2.1.10.3 How to incorporate Custom Tags inside the JSPs

The JSP taglib directive is used to define a tag library and include the custom tags inside the JSPs.

SYNTAX:

<%@ taglib uri="uri of the tag library" prefix="prefix of the tag library" %>

The "**uri**" attribute specifies where to find the tag library description.

The "**prefix**" is unique for the tag library

Example: <%@ taglib prefix="s" uri="/struts-tags"%>

1.2.1.11 Struts 2.x Tags in detail

The Struts 2.x tags can be classified under the following categories.

i) UI Tags
ii) Control Tags
iii) Data Tags

1.2.2 UI Tags

Every element rendered in the browser is implemented as a UI component in Struts. All the UI Tags have following features.

i) State and Behaviour

ii) Rendering Model

iii) Event and Listener Model

iv) Conversion Model

v) Validation Model

UI tags can be grouped into following kind of components

Inputs: Tags for taking input from the user

e.g) Text Field, Text Area, Password etc.

Commands: Tags for performing some action

e.g) Submit, Reset etc.

Selections: Tags used for displaying options to the user Single or multiple values can be selected depending upon the tag used

e.g) Checkbox, Checkbox list, select, radio,doubleselect,combobox etc.

Others: Tags such as forms, messages and components that lay out other components

e.g) form,actionerror,actionmessage,div,tabbedPannel etc.

List of few UI Tags

SI No	**Tag Name**	**Description**
1	Form	HTML form
2	TextField	Single-line text input
3	TextArea	Multiline text input
4	Password	Password input
5	Hidden	Hidden field
6	Radio	Radio Button
7	Select	Drop down list
8	Check box	Check box
9	Check box list	Similar to select tag.
10	Submit	Submit button
11	Doubleselect	Link together to select boxes.
12	Label	Label for another component
13	Actionerror	Render the action errors
14	Actionmessage	Render the action messages.

Few common attributes of UI tags

SI No	Attribute Name	Description
1	Id	Unique identifier of the tag

2	name	Name which we gave for the tag
3	Value	A component's value.
4	required	A Boolean, if true, mandates a value to be entered for the component. Default is false
5	CssStyle	Specifies inline style for the component
6	CssClass	CSS style class name

1.2.2.1 Sample Scenario 1

Problem Description: Develop a Struts 2 application to accept the user's name as input and on submitting the form, display a welcome message or a failure message to the user on the respective pages based on the entered user name.

Step 1: Create a Dynamic Web project ValidateProject

Step 2: Configure the required JAR files as explained in the previous example

Step 3: Configure the web.xml by adding the filter as explained in the previous example

Step 4: Crate an action class UserAction under com.test.user package as given below.

```java
package com.test.user;

public class UserAction {

    private String userName;
    //setters and getters

    public String execute()
    {
        if(this.userName.equals("JOE"))

    {
        return "success";
    }
    else
    {
        return "fail";
    }
    }
```

Step 5: Crate an action class UserAction under com.test.user package as given below.

Step 6: Create a JSP UserInfo.jsp inside a jsp folder under web content folder of your project. And add the following code inside as given below.

//add your JSP code here.

Step 7: Configure the struts.xml as given below.

//add strut.xml code here

Step 8: Test your application

1.2.2.2 Sample Scenario 2

Objective: To get familiarized with various UI Components

Scenario: Develop a struts 2.x application for validating the login credentials of a user. If the User enters a valid user id and password, he/she should be able to navigate to home page of the application else display an appropriate error message.

Step 1: Create a dynamic web project

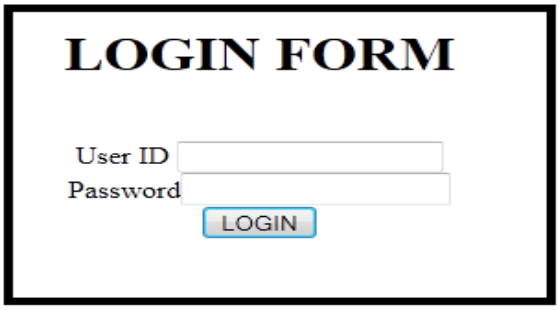
Step 2: Configure the required jar files as explained in the previous examples

Step 3: Configure the web.xml by adding the filter as explained in the previous example

Step 4: Create a JSP page Login.jsp and add the following components inside the JSP.

Form Field	Appropriate UI tag to be Used
User ID	TextField
Password	Password
Labels	For user id and password
Login Button	Submit Button

Login.jsp should look like below



The code snippet for designing this UI is shown below.

```
<%@ taglib uri="/struts-tags"  prefix="s"%>

<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">

<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>Insert title here</title>
</head>
<body>

<s:form action="authenticate" theme="simple">

  <center>
          <h1>LOGIN FORM</h1><br>

          <s:label>User ID</s:label>  <s:textfield name="userId"></s:textfield><br>

          <s:label>Password</s:label><s:password name="password"></s:password><br>

          <s:submit value="LOGIN"></s:submit>
  </center>
  </s:form>
```

Note:

In the above code snippet, you can see that, UI tags like, form,textfield,label etc. are written as <s:form>,<s:textfield>,<s:label> etc. Here the character 's' indicates the value of the prefix attribute which we used inside the tag library. It indicates that, whatever tags preceeding with 's' belongs to the custom tag library mentioned in the URI path as below.

<%@ taglib uri="*/struts-tags*" prefix="*s*"%>

Step 5: Create the Action class named LoginAction.java under com.test.login package.

```java
package com.test.login;
public class LoginAction {

    private String userId;
    private String password;

    //setters and getters

```

```java
public String execute()
{
    if(this.getUserId().equals("admin")&& this.getPassword().equals("admin"))
    {
        return "success";
    }
    else
    {
        return "fail";
    }
}
```

Note that, the above action class contains two fields, userId and password. They will hold values passed from form, and also contains an execute() method that will authenticate users.That method will return a String value ("Success" or "Fail") which is mapped as the result name inside action tag of struts.xml.Based on this value returned, navigation will happen either to Success.jsp or Fail.jsp.

Step 6: Configure Struts.xml

```xml
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE struts PUBLIC
"-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
"http://struts.apache.org/dtds/struts-2.0.dtd">
<struts>
<constant name="struts.enable.DynamicMethodInvocation" value="false" />
<constant name="struts.devMode" value="true" />
```

```
<include file="example.xml"/>
<package name="test" namespace="/" extends="struts-default">
<default-action-ref name="sayHello"/>
<action name="authenticate" class="com.test.login.LoginAction">
<result name="success">/jsp/Success.jsp</result>
<result name="fail">/jsp/Fail.jsp</result>
</action>
</package>
</struts>
```

Here in the struts.xml, value of the name attribute of the action tag should be same as the value of the action attribute of the form tag in your JSP pages.

In this scenario,struts framework identifies the action name as authenticate from Login.jsp and search for the same under action tag in struts.xml.Here it identifies the action class name as com.test.login.LoginAction.Since we haven't specified method name for the action class,framework will search for the default method execute() inside Login Action.

Step 7:

i) Create a Success.jsp page to display a Welcome message when the user enters right credentials.

ii) Create a Fail.jsp page to display an error message, when the user enters wrong credentials.

Step 8: Test your application

1.2.2.3 Sample Scenario 3

Objective: To get familiarized with various UI Components

Scenario: TCS Bank plans to provide an online facility for its customers to open an Account. Following are the requirements for this functionality. The application should accept the following details from the user: Customer Name, Date of Birth, Gender, Address, E-Mail, Type of Account, Services. Type of Account can be Savings/Current. User can choose to avail any of these services: Mobile Alerts, Debit Card. On submission of the form the user should be displayed success page showing the summary of the details entered by the user.

Step 1: Create dynamic web project

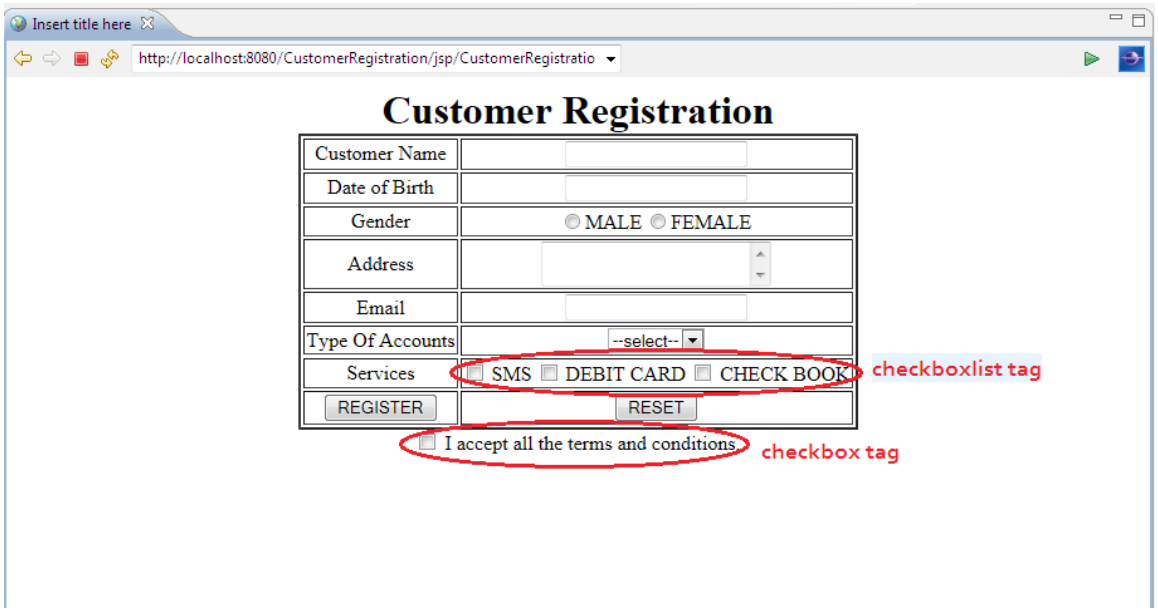
Step 2: Configure the jar files

Step 3: Configure the web.xml

Step 4: Create jsp page CustomerRegistration.jsp and add the UI Components from the palette to accept the input from the user

SI NO	Form Field	Appropriate Tag to be Used
1	Customer Name	TextField
2	Date of Birth	Text Field
3	Gender	Radio Button
4	Address	Text Area
5	Email	Text Field
6	Type Of Account	Select(drop down box)
7	Services	CheckBoxlist

The Customer Registration page should look like as given below,



Step 5: Add the following code in side CustomerRegistration.jsp

```
<%@taglib uri="/struts-tags" prefix="s"%>
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
pageEncoding="ISO-8859-1"%>
<%@taglib uri="/struts-tags" prefix="s"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>Insert title here</title>
</head>
<body>
<center>
<h1>Customer Registration</h1>
<table border="2">

<s:form action="customerregister" theme="simple">
<tr >
<td><s:label value="Customer Name"/></td>
<td><s:textfield name="customerName"/></td>
</tr>
<tr>
<td><s:label value="Date of Birth"/></td>
<td><s:textfield name="dateofBirth"/></td>
</tr>
<tr>
<td><s:label value="Gender"/></td>
<td> <s:radio name="gender" list="{'MALE','FEMALE'}" /> </td>
</tr>
<tr>
<td><s:label value="Address"/></td>
<td> <s:textarea name="address"/> </td>
</tr>
<tr>
<td><s:label value="Email"/></td>
```

```
<td> <s:textfield name="email"/> </td>
</tr>
<tr>
<td><s:label value="Type Of Accounts"/></td>
<td> <s:select list="{'--select--','Savings','Current'}" name=" ac-
countType" ></s:select> </td>
</tr>
<tr>
<td>
<s:label value="Services"></s:label>
</td>
<td>
<s:checkboxlist name="services"
list="{'SMS','DEBIT CARD','CHECK BOOK'}" />
</td>
</tr>
<tr>
<td><s:submit value="REGISTER"/></td>
<td> <s:submit value="RESET"></s:submit> </td>
</tr>
</s:form>
</table>

<s:checkbox name="receiveUpdates" fieldValue="true" label="I accept all the terms
and conditions."></s:checkbox>
</center>
</body>
</html>
```

i) In Radio tag, the list attribute is used to specify the option values, The value of the list attribute can be a Collection, Map, Array .Here we use Array.

```
<s:radio name="gender" list="{'MALE','FEMALE'}" />
```

ii) In Select tag the list attribute is used to specify the option values.The value of the list attribute can be a Collection,Map,or Array.Here we used arrays for static population. ie. We hard coded the values.

```
<s:select list="{'--select--','Savings','Current'}" name=" accountType" />
```

Few commonly used attributes of select tags are,

1)list

2) listKey

3) listValue

4) headerKey

5) headerValue

iii) The Checkbox tag returns a boolean value. If the checkbox is checked then true is returned else false is returned

```
<s:checkbox name="receiveUpdates" fieldValue="true" label="I accept all the terms and conditions."/>
```

iv) The checkboxlist tag is same as that of the select tag, the only difference is that it displays boxes for each option instead of a dropdown. It returns an array of String values.

```
<s:checkboxlist name="services" list="{'SMS','DEBIT CARD','CHECK BOOK'}" />
```

Step 4: Create a Action class CustomerAction.java for registering the customer. It will have properties corresponding to each input field of the CustomerRegistration form. For UI components which allow multiple selections like check-box/checkboxlist here, the action class should have a property of collection type (array/list) for storing the set of data selected by the user (different options selected by the user).

The Action class for the Customer Registration form is as shown below:

```
package com.test.customer;

public class CustomerAction {
private String customerName;
```

```
private String dateofBirth;
private String email;
private String gender;
private String address;
private String accountType;
private String[]services; // property for checkbox

//getters and setters for above properties to be included

}
```

Step 5:

Add navigation rule with a navigation case corresponding to the form tag's action attribute inside struts.xml. Here the action attribute value will be “customerregister”. On submit the user should be navigated to success page (RegistrationSuccess.jsp) which should display the details entered by the user in the CustomerRegistration form as shown below.

```
<package name="test" namespace="/" extends="struts-default">
<action name="customerregister" class="com.test.customer.CustomerAction">
<result name="success">/jsp/RegistrationSuccess.jsp.jsp</result>
</action>
</package>
```

Step 6:

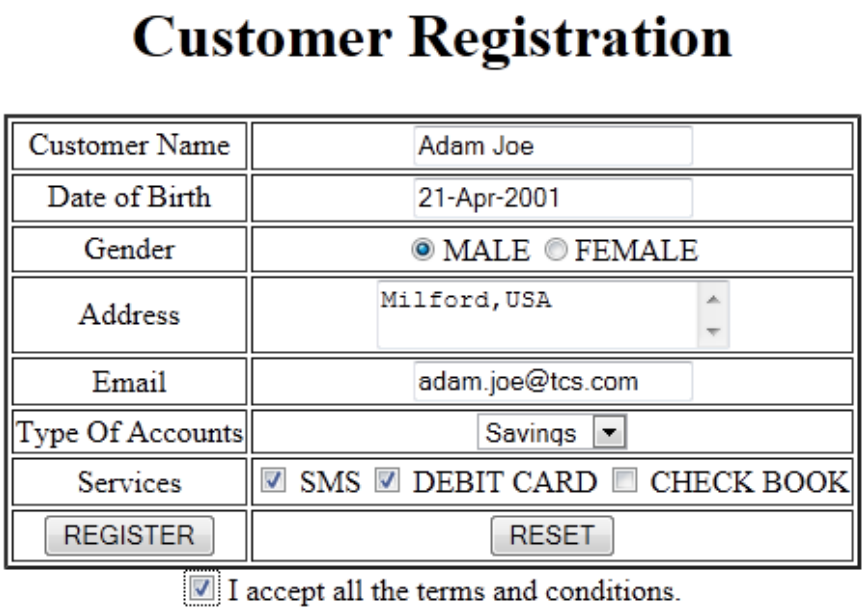
Create RegistrationSuccess.jsp and display all the details entered by the user using <s:property> tag as shown below.

```
<%@taglib uri="/struts-tags" prefix="s"%><%@ page language="java" contentType="text/html; charset=ISO-8859-1"
pageEncoding="ISO-8859-1"%>
<%@taglib uri="/struts-tags" prefix="s"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
```

```
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>Insert title here</title>
</head>
<body>
<s:form theme="simple">
<center>
<h1>Customer Registration</h1>

<table border="2">
<tr >
<td><s:label value="Customer Name"/></td>
<td><s:property value="customerName"/></td>
</tr>
<tr>
<td><s:label value="Date of Birth"/></td>
<td><s:property value="dateofBirth"/></td>
</tr>
<tr>
<td><s:label value="Gender"/></td>
<td> <s:property value="gender"/> </td>
</tr>
<tr>
<td><s:label value="Address"/></td>
<td> <s:property value="address"/> </td>
</tr>
<tr>
<td><s:label value="Email"/></td>
<td> <s:property value="email"/> </td>
</tr>
<tr>
<td><s:label value="Type Of Accounts"/></td>
<td> <s:property value="accountType"/> </td>
</tr>
<tr>
<td><s:label value="Services" /></td>
<td><s:property value="services"/></td>
```

```
</tr>
</table>
</center>
</s:form>
</body>
</html>
```

Step 7: Test your application



Customer Registration

Customer Name	Adam Joe
Date of Birth	21-Apr-2001
Gender	(•) MALE ( ) FEMALE
Address	Milford,USA
Email	adam.joe@tcs.com
Type Of Accounts	Savings
Services	[x] SMS [x] DEBIT CARD [ ] CHECK BOOK
REGISTER	RESET

[x] I accept all the terms and conditions.

Customer Registration

Customer Name	Adam Joe
Date of Birth	21-Apr-2001
Gender	MALE
Address	Milford,USA
Email	adam.joe@tcs.com
Type Of Accounts	Savings
Services	SMS, DEBIT CARD

1.2.2.4 Sample Scenario 4

Objective: To populate a select menu (drop down) dynamically

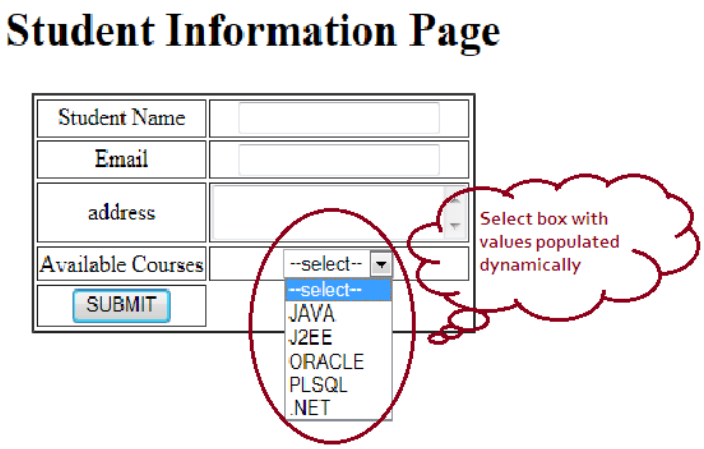
Problem Scenario: ABC InfoTech plans to provide an online facility for its admin to register the student details .Following are the requirements for this functionality. The form should accept student name, Email, Address and all the available courses (should be a list: drop down) which he can enrolled for. On the successful submission, success message needs to be displayed.

Step 1: Create a dynamic web project StudentTestProject

Step 2: Configure the Jar files

Step 3: Configure web.xml

Step 4: Create jsp page StudentInfo.jsp as per the design shown below (use Appropriate UI components)



Step 5: Create a Action class StudentAction.java in package com.test.student.StudentAction for Student Information Form. It will have properties corresponding to each input field of the form and the output fields to be displayed on the student details page. The action class for the student Information form is as shown below:

```
package com.test.student;

import java.util.ArrayList;

public class StudentAction {

    private String studentName;
    private String email;
    private String address;
    private ArrayList<Course> courseList;

    private String course;

    //corresponding setters and getters method

}
```

A property of ArrayList<Course> type is required for populating a select UI component dynamically

Step 5: Add the following two methods in side the StudentAction class as given below.

```
public String populateDropDown()
{
   courseList=new ArrayList<Course>();
   courseList.add(new Course("JAVA","JAVA"));
   courseList.add(new Course("J2EE","J2EE"));
   courseList.add(new Course("ORACLE","ORACLE"));
   courseList.add(new Course("PLSQL","PLSQL"));
   courseList.add(new Course(".NET",".NET"));
   return "populate";

}
```

```
public String execute()
{

   return "success";
}
```

Note: In the above methods, populateDropDown() method contain the code for populating the drop down box dynamically and return a string value "populate".The other method is execute which is as usual returns a string "success".

```
<package name="test" namespace="/" extends="struts-default">

   <default-action-ref name="populateAction"/>

<action name="populateAction" class="com.test.student.StudentAction"  method="populateDropDown" >
   <result name="populate">/jsp/StudentInfo.jsp</result>
   </action>

    <action name="student" class="com.test.student.StudentAction" >
   <result name="success">/jsp/StudentDetails.jsp</result>
</action>
</package>
```

Step 6: Configure the struts.xml as given below.

In struts.xml,

Section I

i)Set the name for default-action-ref tag as "populateAction".

ii)In side the action tag,set the the value of name attribute as "populateAction".

iii)populateAction is bound with the populateDropDown() method inside the StudentAction class.

iv)In this scenario, populateAction is the default action.

v)populateDropDown() method will be automatically invoked and then redirected to the form(StudentInfo.jsp).

Section 2

i) Set the action name as "student" in the second action tag.

ii) Map the action name against class com.test.student.StudentAction

iii) execute method inside the StudentAction will be invoked automatically as it is the default method.

iv) Based on the return type ,response will be redirected to StudentDetails.jsp.

Step 7: Create a JSP page ,StudentInfo.jsp and add the necessary fields for Student Name,Email,Address and for displaying the available courses in the drop down.

To populate drop down box dynamically, add the following code inside the StudentInfo.jsp.

```
<tr>
<td><s:label value="Available Courses"></s:label></td>
<td><s:select list="courseList" name="course" listKey="courseId" listValue="courseName" headerKey="--select--" headerValue="--select--"></s:select></td>
</tr>
```

Here we display the country dropdown using the select tag.

The option values of the drop down box are using the courseList property of the StudentAction class.

The courseList is of type ArrayList and contain values of type Course.

The Course class has courseId and courseName as attributes.

i) list attribute of select tag contains the values of the courseList.

ii) listKey attribute holds the values of the value to be stored in the backend.

In this scenario, course id is stored in the back-end.

iii) listValue attribute holds the value to be displayed in the front-end. In this scenario, couse name is stored in the front-end.

iv) name attribute holds the value of the option which we select from the drop down.

For example, if there is a drop down box of accounts, with account no as listKey, account type as listValue.

1001 SAVINGS

1002 CUURENT

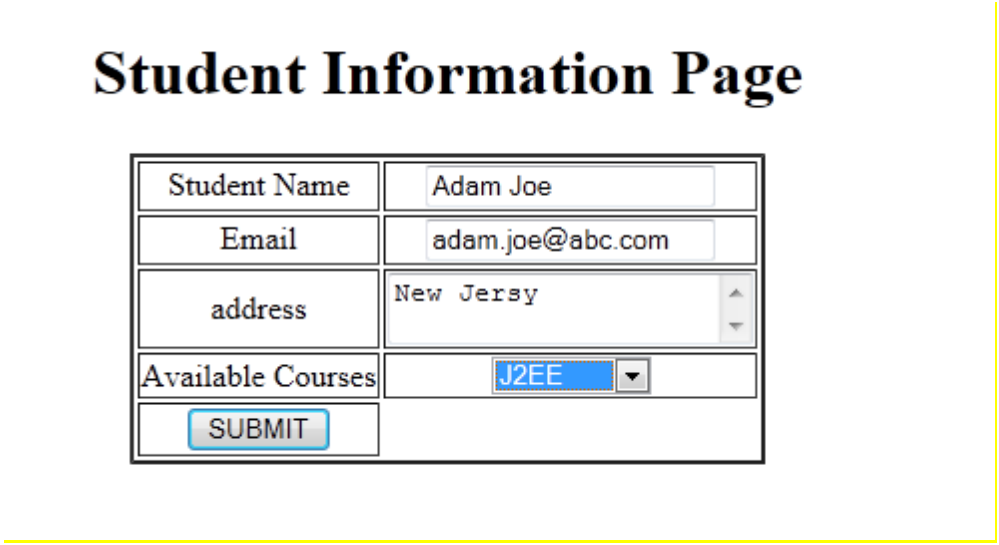
1003 LOAN

In this case account type is displayed in the front-end. If we select SAVINGS from the drop down , account no ie 1001 will be stored in the back end. Ie the 1001 will be assigned in the name attribute.

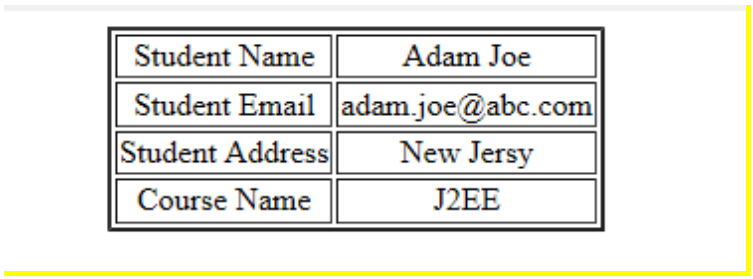
v) headerValue attribute can be used to specify the the first value in the drop down box.

vi) headerKey attribute can be used to specify the corresponding key value.

Step 8: Test your application



On click of Submit ,



Student Name	Adam Joe
Student Email	adam.joe@abc.com
Student Address	New Jersy
Course Name	J2EE

1.2.2.5 Control Tags

Struts 2 provide a set of control tags to control flow of the application. Some of the important control tags are

i) If Else
ii) Iterator
iii) Merge
iv) Sort
iv) Generator

1.2.2.6 If Else Tag with Scenario

This tag is useful for executing the simple if or else condition by taking the values from a variable.

Scenario 1:

The following example will prompts the user to select a state from the list and displays the language of the state in the next page.

Step1: Create a dynamic project

Step 2: Configure the jar files

Step 3: Configure web.xml

Step 4: Create a class StateAction.java under com.test.state package.

```
package com.test.state;
public class StateAction {
private String state;

public String getState() {
return state;
}

public void setState(String state) {
this.state = state;
}
public String execute()
{
return "success";
}
```

Step 5: Create a JSP State.jsp and add the following code inside.

```
<s:form action="stateAction" theme="simple">
<s:label value="States"></s:label>
<s:select list="{'KERALA','TAMILNADU','UP','GUJARAT'}" name="state"
headerKey="--select--" headerValue="--select--"></s:select>
<s:submit value="SELECT"></s:submit>
</s:form>
```

Step 6: Configure struts.xml

```
<package name="test" namespace="/" extends="struts-default">
<default-action-ref name="defaultView"/>
<action name="defaultView" >
<result name="success">States.jsp</result>
</action>
<action name="stateAction" class="com.test.state.StateAction">
<result name="success">DisplaySelectedState.jsp</result>
</action>
</package>
```

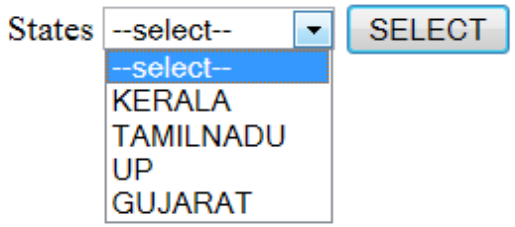
Step 4:

Create a JSP DisplaySelectedState.jsp and add the given below code inside it.

```
<s:if test="state=='KERALA'">
<s:label value="You Have Selected "></s:label>
<s:label name="state"></s:label>

<s:label value=" Language is MALAYALAM"></s:label>
</s:if>

<s:elseif test="state=='TAMILNADU'">
<s:label value="You Have Selected "></s:label>
<s:label name="state"></s:label>
<s:label value=" Language is TAMIL"></s:label>
</s:elseif>

<s:elseif test="state=='UP'">
<s:label value="You Have Selected "></s:label>
<s:label name="state"></s:label>
<s:label value=" Language is HINDI"></s:label>
```

```
</s:elseif>

<s:elseif test="state=='GUJARAT'">
<s:label value="You Have Selected "></s:label>
<s:label name="state"></s:label>
<s:label value=" Language is GUJARATI"></s:label>
</s:elseif>
<s:else>
<s:label value=" Please select a state"></s:label>
</s:else>
```

Step 5: Test your application



You Have Selected TAMILNADU Language is TAMIL

1.2.2.7 Iterator Tag with Scenario

Struts 2 iterator tag is useful for iterating a value which of type java.util.Collections or Iterator.

Scenario

The following example will print the details of all employees using iterator tag.

Step 1:Create a dynamic web project

Step 2:Configure the jar files and web.xml

Step 3: Create a class Employee with attributes, empId,empName and salary under com.test.iterator package.

```
package com.test.iterator;

public class Employee {

    private int empId;
    private String empName;
    private double salary;
    public Employee(int empId, String empName, double salary) {
        super();
        this.empId = empId;
        this.empName = empName;
        this.salary = salary;
    }

    // setters and getters
```

Step 4: Create a class EmployeeAction with an attribute empList basically an ArrayList containing Employee object under com.test.iterator package.

```java
package com.test.iterator;
import java.util.ArrayList;
public class EmployeeAction {
    private ArrayList<Employee>empList;
    public ArrayList<Employee> getEmpList() {
        return empList; }
    public void setEmpList(ArrayList<Employee> empList) {
        this.empList = empList;
    }
    public String execute()
    {
        empList=new ArrayList<Employee>();
        empList.add(new Employee(1001, "Sam", 20000));
        empList.add(new Employee(1002, "Allen", 30000));
        empList.add(new Employee(1003, "Richie", 10000));
        empList.add(new Employee(1004, "Parker", 50000));

        return "success";
    }
}
```

Step 5: **Configure the struts.xml**

Step 6: **Create a JSP EmployeeDetails.jsp and add the below given code inside.**

```html
<s:form theme="simple">
<h1>Employee Report Using Iterator Tag</h1>
<table border="2">
<tr><td>Employee ID</td><td>Employee Name</td><td>Salary</td>
<s:iterator value="empList">
<tr>
<td><s:property value="empId"/> </td>
<td><s:property value="empName"/></td>
<td><s:property value="salary"/></td>
</tr>
</s:iterator>
</table>
</s:form>
```

Step 7: **Test your application**

Employee Report Using Iterator Tag

Employee ID	Employee Name	Salary
1001	Sam	20000.0
1002	Allen	30000.0
1003	Richie	10000.0
1004	Parker	50000.0

1.2.2.8 Merge Tag with Scenario

Merge tag is using for merging multiple lists into a single list and use it in your view .The first elements from all the lists are merged first, then second element and so on .Merge tag has a nested param tag using for merging the lists. You can use as many param tags inside merge tag to merge multiple lists.

Scenario

The following example will explain the working of merge tag.

In this example, we have two different lists containing the store manager details of different regions (North, and South). We need to print the single report of all the store managers' details from all the regions.

Step 1: Create a dynamic web project

Step 2: Configure the jar files and web.xml

Step 3: Create a class Manager having properties manager name, manager id, store,region under com.test.merge package.

```java
public class Manager {

    private String managerName;
    private String store;
    private int managerId;
    private String region;

    public Manager(String managerName, String store, int managerId,String region)
    {

        this.managerName = managerName;
        this.store = store;
        this.managerId = managerId;
        this.region = region;
    }
    //setters and getters
```

Step 4: create a class RegionAction having two list having manager details of two different regions under com.test.merge package.

```java
import java.util.ArrayList;

public class RegionAction {

    private ArrayList<Manager>northManagerList;
    private ArrayList<Manager>southManagerList;

    //setters and getters

    public String execute()
    {
        northManagerList=new ArrayList<Manager>();
        southManagerList=new ArrayList<Manager>();

        southManagerList.add(new Manager("SAM", "CHENNAI", 101, "SOUTH"));
        southManagerList.add(new Manager("JOE", "TRIVANDRUM", 102, "SOUTH"));
        southManagerList.add(new Manager("SAM", "HYDERABAD", 103, "SOUTH"));

        northManagerList.add(new Manager("ALLEN", "DELHI", 104, "NORTH"));
        northManagerList.add(new Manager("MIKE", "MUMBAI", 105, "NORTH"));
        northManagerList.add(new Manager("RICHIE", "KOLCUTA", 106, "NORTH"));
        return "success";
    }
```

Step 5: **configure struts.xml**

Step 6: create a JSP RegionStoreManagerReport.jsp and add the following code inside the JSP page.

```
<s:iterator value="managersList">
<TR>
<td><s:property value="managerName" /></td>
<td><s:property value="managerId"/> </td>
<td><s:property value="store"/> </td>
<td><s:property value="region"/> </td>
</TR>
</s:iterator>
```

Step 7: **Test your application**

SPEC ENTERPRISES LTD

ALL REGION STORE MANAGER REPORT

MANAGER NAME	MANAGER ID	STORE	REGIONS
ALLEN	104	DELHI	NORTH
SAM	101	CHENNAI	SOUTH
MIKE	105	MUMBAI	NORTH
JOE	102	TRIVANDRUM	SOUTH
RICHIE	106	KOLCUTA	NORTH
SAM	103	HYDERABAD	SOUTH

Note that

First we merged two lists, southManagerList and southManagerList into one list managerList. And then we were iterating through the main list managerList through iterator tag.

1.2.2.9 Sort Tag with Scenario

One of the most important control tag is sort. Sort tag is used to sort a List using a java.util.Comparator.

Scenario

The following example will explain the working of sort tag.

In the previous example we have already generated the report of the entire manager from different regions. Now we need to sort the report by manager id and store using sort tag.

Step 1: Create a class Manager having properties manager name,manager id,store,region under com.test.sort package.

```
public class Manager {

    private String managerName;
    private String store;
    private int managerId;
    private String region;

    public Manager(String managerName, String store, int managerId,String region)
    {

        this.managerName = managerName;
        this.store = store;
        this.managerId = managerId;
        this.region = region;
    }
    //setters and getters
```

Step 2: create a class RegionAction having a list manager details of two different regions under com.test.sort package.

```java
import java.util.ArrayList;

public class RegionAction {

    private ArrayList<Manager>managerList;


    //setters and getters

    public String execute()
    {
        managerList=new ArrayList<Manager>();

        managerList.add(new Manager("SAM", "CHENNAI", 101, "SOUTH"));
        managerList.add(new Manager("ALLEN", "DELHI", 104, "NORTH"));
        managerList.add(new Manager("JOE", "TRIVANDRUM", 102, "SOUTH"));
        managerList.add(new Manager("MIKE", "MUMBAI", 105, "NORTH"));
        managerList.add(new Manager("RICHIE", "KOLCUTA", 106, "NORTH"));
        managerList.add(new Manager("SAM", "HYDERABAD", 103, "SOUTH"));
        return "success";
    }
```

Step 3: **create a class ManagerIDComparator which implements from Comparator interface under com.test.sort package. Add the following code inside the class.**

```java
import java.util.Comparator;

public class ManagerIDComparator implements Comparator<Manager> {

    public int compare(Manager m1, Manager m2) {

        return m1.getManagerId()-m2.getManagerId();
    }
}
```

Step 4: **Create a class StoreComparator implements from Comparator interface under com.test.sort package. Add the following code inside.**

```java
import java.util.Comparator;

public class StoreComparator implements Comparator<Manager> {
    public int compare(Manager m1, Manager m2) {
        // TODO Auto-generated method stub
        return m1.getStore().compareTo(m2.getStore());
    }

}
```

Step 5: **Confgure struts.xml.**

Step 6: **Create a JSP RegionStoreManagerReport.jsp and add the following code inside**

```html
<h2>Un Sorted Manager's Report</h2>
<table border="2">
<tr>
<td>MANAGER NAME</td><td>MANAGER ID</td><td>STORE</td><td>REGIONS</td>
</tr>
<s:iterator value="managerList">
<TR>
<td><s:property value="managerName" /></td>
<td><s:property value="managerId"/> </td>
<td><s:property value="store"/> </td>
<td><s:property value="region"/> </td>
</TR>
</s:iterator>
```

```html
<h2>Sorted Manager's Report By Manager ID</h2>
<table border="2">

<tr>
<td>MANAGER NAME</td><td>MANAGER ID</td><td>STORE</td><td>REGIONS</td>
</tr>
<s:bean name="com.test.sort.ManagerIDComparator" var="mgrID"/>
<s:sort comparator="#mgrID" source="managerList">
<s:iterator>
<tr>
<td><s:property value="managerName" /></td>
<td><s:property value="managerId"/> </td>
<td><s:property value="store"/> </td>
<td><s:property value="region"/> </td>
</tr>
</s:iterator>
</s:sort>
```

```
<h2>Sorted Manager's Report By Store</h2>
<table border="2">

<tr>
<td>MANAGER NAME</td><td>MANAGER ID</td><td>STORE</td><td>REGIONS</td>
</tr>
<s:bean name="com.test.sort.StoreComparator" var="str"/>
<s:sort comparator="#str" source="managerList">
<s:iterator>
<tr>
<td><s:property value="managerName" /></td>
<td><s:property value="managerId"/> </td>
<td><s:property value="store"/> </td>
<td><s:property value="region"/> </td>
</tr>
</s:iterator>
</s:sort>
```

Step 7: Test your application.

SPEC ENTERPRISES LTD

ALL REGION STORE MANAGER REPORT

Un Sorted Manager's Report

MANAGER NAME	MANAGER ID	STORE	REGIONS
SAM	101	CHENNAI	SOUTH
ALLEN	104	DELHI	NORTH
JOE	102	TRIVANDRUM	SOUTH
MIKE	105	MUMBAI	NORTH
RICHIE	106	KOLCUTA	NORTH
SAM	103	HYDERABAD	SOUTH

Sorted Manager's Report By Manager ID

MANAGER NAME	MANAGER ID	STORE	REGIONS
SAM	101	CHENNAI	SOUTH
JOE	102	TRIVANDRUM	SOUTH
SAM	103	HYDERABAD	SOUTH
ALLEN	104	DELHI	NORTH
MIKE	105	MUMBAI	NORTH
RICHIE	106	KOLCUTA	NORTH

Sorted Manager's Report By Store

MANAGER NAME	MANAGER ID	STORE	REGIONS
SAM	101	CHENNAI	SOUTH
ALLEN	104	DELHI	NORTH
SAM	103	HYDERABAD	SOUTH
RICHIE	106	KOLCUTA	NORTH
MIKE	105	MUMBAI	NORTH
JOE	102	TRIVANDRUM	SOUTH

1.2.2.10 Generator Tag with Scenario

We can use generator tag to create an iterator from given values. We can provide separator to use the delimiter for the values

```
<s:generator val="%{'RED,GREEN,BLUE'}" separator="," var="intsIterator" />

<s:iterator value="#intsIterator">

    <s:property /><br/>

</s:iterator>
```

Scenario:

The example given below, will display the list of cities on the selection of respective states.

Step 1: Create a dynamic web project
Step 2: Configure the jar files and web.xml
Step 3: Create a class StateAction under com.test.state package as follows

```
package com.test.state;
public class StateAction {
private String stateName;

public String getStateName() {
return state;
}

public void setStateName(String state) {
this.state = state;
}
public String execute()
{
return "success";
}
```

Step 4: Configure struts.xml as given below.

```
<default-action-ref name="defaultView"/>
```

```
<action name="defaultView" class="com.test.generator.StateAction" >
<result name="success">/generator/States.jsp</result>
</action>
<action name="stateAction" class="com.test.generator.StateAction" >
<result name="success">/generator/ShowCities.jsp</result>
</action>
```

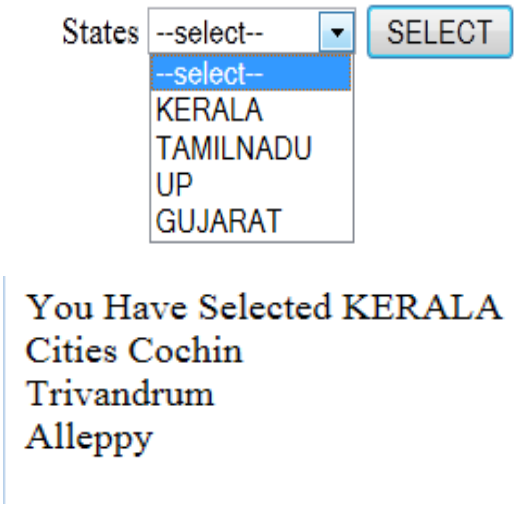
Step 5: **Design a JSP page, State.jsp and add the given below code inside.**

```
<s:form action="stateAction" theme="simple">

<s:label value="States"></s:label>

<s:select list="{'KERALA','TAMILNADU','UP','GUJARAT'}" name="stateName" headerKey="--select--"
headerValue="--select--">

</s:select>
<s:submit value="SELECT"></s:submit>
</s:form>
```

Step 6: **Design a JSP page ShowCities.jsp and add the given below code inside.**

```
<s:form theme="simple">
<s:if test="stateName=='KERALA'">
<s:label value="You Have Selected "></s:label>
<s:label name="stateName"></s:label><br>
    <s:generator val="%{'Cochin','Trivandrum','Alleppy'}"separator=",">
       <s:iterator>
              <s:property /><br/>
       </s:iterator>
    </s:generator>


</s:if>
```

Step 7: **Test your application**



1.2.2.11 Data Tags

Data tags are used for manipulate the data displayed on the page. Important data tags are

a) property tag
b) set tag
c) push tag
d) bean tag
e) action tag
f) include tag
g) param tag

1.2.2.12 Property Tag

Most commonly used data tag. Property tag is using for getting the value from the ValueStack or from different scopes (application, session or request etc.).

Some important attributes of the property tag are

Value: to holds the value from the value stack.

Default: holds the default value, if the value stack is returning a null value.

e.g)<property value=”course” default=”JAVA”/>

1.2.2.13 Set Tag

Assign a value to a variable in a specific scope.

```
eg) <s: set name=”var” value=”value1”/>
<s:property value=”var”/>
```

1.2.2.14 Bean Tag

Bean Tag used for assigning a bean reference to a JSP page.Using <s:param> tag we can pass the values to the bean reference and set the value.

eg)

```
<s:bean name="com.test.datatags.Person" var="per">

<s:param name="name" >BOB SAMUEL</s:param>

<s:param name="address" >NEW JERSY</s:param>

</s:bean>
```

1.2.2.15 Push Tag

Push tag is using for pushing the values to the top of the value stack. Once the push tag pushes the values to the top, we don’t require the bean reference to access the values.

```
<s:push value="#per">
Person Name<s:property value="name"/><br>
Person Address <s:property value="address"/>
</s:push>
```

1.2.2.16 Include Tag

Include tag is using for including the contents of HTML page or JSP page in the current JSP page.

<s:include value="URL of the page to be included"/>

1.2.2.17 Param Tag

Please refer the bean tag section (1.2.6.3.3)

1.2.2.18 Putting all together (Scenario)

Scenario: **The following example will explain the functionalities of various data tags.**

Step 1: **Create a dynamic web project**

Step 2: **Configure the jar files and web.xml**

Step 3: **Create a class Person under com.test.datatags package as given below.**

```
public class Person {
private String name;
private String address;
public String getName() {
    return name;
}
public void setName(String name) {
    this.name = name;
}


public String getAddress() {
    return address;
}
public void setAddress(String address) {
    this.address = address;
}
}
```

Step 4: Create a class PersonAction under com.test.datatags as given below.

```java
public class PersonAction {

    private Person person;

    public Person getPerson() {
        return person;
    }

    public void setPerson(Person person) {
        this.person = person;
    }

    public String execute()
    {
        return "success";
    }
}
```

Step 5: **Configure struts.xml**

Step 6: **Design a JSP two JSPs PersonDetails.jsp and PersonInfo.jsp and the add the following code in side those files.**

PersonDetails.jsp

```jsp
<%-- populating the bean object using bean tag --%>

<s:bean name="com.test.datatags.Person" var="per">
<s:param name="name" >BOB SAMUEL</s:param>
<s:param name="address" >NEW JERSY</s:param>

</s:bean>
```

```
<%--setting the value using set tag through bean tag --%>

<s:set name="personName" value="#per.name"></s:set>
<s:set name="personAddress" value="#per.address"></s:set>

<%-- getting the details of person using property tag --%>

<center>
<h3>Getting person details using bean tag </h3>
Name  : <s:property value="personName"/><br>
Address  : <s:property value="personAddress"/><br>

<%-- setting the details of the person in session scope. --%>

<s:set name="personName" value="#per.name" scope="session"></s:set>
<s:set name="personAddress" value="#per.address" scope="session"></s:set>

<%-- including the contents of PersonInfo.jsp using include tag --%>

<h3>Printing the contents of PersonInfo.jsp</h3>
<s:include value="PersonInfo.jsp">

</s:include>
```

PersonInfo.jsp

```
Person Name <s:property value="#session['personName']"/><br>
Person Address <s:property value="#session['personAddress']"/>
```

Getting person details using bean tag

Name : BOB SAMUEL
Address : NEW JERSY

Getting person details using push tag

Person NameBOB SAMUEL
Person Address NEW JERSY

Printing the contents of PersonInfo.jsp

Person Name BOB SAMUEL
Person Address NEW JERSY

1.2.2.19 Recap

From the previous sections you have understood the following points.

- Developing a web application using struts 2
- Struts 2 life cycle flow.
- Different types of tags used for developing a struts application
- Different types UI tags and their implementation
- Different types of Control tags and their implementation
- Different types of Data tags and their implementation